

# TRABALHO 4 EDA1

## Algoritmos de Busca

ALUNO: RODRIGO DUTRA FERREIRA;

MATRÍCULA: 242015951;

código-fonte completo, explicação resumida da solução e resultados dos testes realizados.

Arquivo de dados: Lista\_Municipios\_com\_IBGE\_Brasil\_Versao\_CSV.csv

Fonte: <https://blog.mds.gov.br/redesuas/lista-de-municipios-brasileiros/>

Estrutura do CSV (separador ponto-e-virgula): IBGE ; Municipio ; UF ; Regiao ; Populacao ; Porte

O arquivo esta ordenado pelo campo IBGE (inteiro), o que permite aplicar busca binaria diretamente sobre o vetor de indices.

CÓDIGO-FONTE COMPLETO:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

#define NOME_ARQUIVO "Lista_Municipios_com_IBGE_Brasil_Versao_CSV.csv"

#define TAM_CAMPO    128

/* Tipos */

/*
 * Indice: unico vetor mantido em memoria.
 *
 * ibge -> codigo IBGE7 (7 digitos), chave de busca
 *
 * linha -> numero da linha no arquivo (cabecalho = 1, primeiro dado = 2)
 */

typedef struct {
    int ibge;
    int linha;
} Indice;

/*
 * Municipio: preenchido sob demanda ao exibir resultado.
 */
```

```

*/

typedef struct {

    int  ibge;

    char municipio[TAM_CAMPO];

    char uf[TAM_CAMPO];

    char regiao[TAM_CAMPO];

    long populacao;

    char porte[TAM_CAMPO];

} Municipio;


/* Prototipos */


int  carregarIndice(const char *nomeArq, Indice **vetor);

int  buscaSequencial(Indice *vetor, int n, int ibge);

int  buscaBinaria(Indice *vetor, int n, int ibge);

int  lerLinha(const char *nomeArq, int numLinha, Municipio *m);

void  exibirMunicipio(const Municipio *m);

void  consultarMunicipio(Indice *vetor, int n);

void  liberarMemoria(Indice **vetor);

double medirTempo(clock_t inicio, clock_t fim);


/* main */


int main(void) {

    Indice *vetor = NULL;

    int      n      = 0;

    int      opcao;

    do {

        printf("\n==== Algoritmos de Busca - Censo IBGE 2010 ==== \n");

        printf("1. Carregar dados \n");

        printf("2. Consultar municipio \n");

```

```
printf("3. Sair\n");

printf("Opcao: ");

scanf("%d", &opcao);

switch (opcao) {

    case 1:

        if (vetor != NULL) {

            liberarMemoria(&vetor);

            n = 0;

        }

        n = carregarIndice(NOME_ARQUIVO, &vetor);

        if (n > 0)

            printf("Dados carregados: %d municipios.\n", n);

        break;

    case 2:

        if (vetor == NULL || n == 0)

            printf("Carregue os dados primeiro (opcao 1).\n");

        else

            consultarMunicipio(vetor, n);

        break;

    case 3:

        liberarMemoria(&vetor);

        printf("Memoria liberada. Encerrando.\n");

        break;

    default:

        printf("Opcao invalida.\n");

}

} while (opcao != 3);

return 0;
```

```

}

/* carregarIndice */
/* Le o arquivo CSV e preenche o vetor com (IBGE7, linha). */
/* Coluna IBGE7 e a terceira (indice 2, base 0). */

int carregarIndice(const char *nomeArq, Indice **vetor) {
    FILE *arq = fopen(nomeArq, "r");

    if (arq == NULL) {
        fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", nomeArq);
        return -1;
    }

    char buf[1024];

    /* contagem de linhas de dados */

    int total = 0;

    fgets(buf, sizeof(buf), arq); /* descarta cabecalho */

    while (fgets(buf, sizeof(buf), arq) != NULL) {
        /* ignora linhas vazias */

        if (buf[0] != '\n' && buf[0] != '\r' && buf[0] != '\0')
            total++;
    }

    *vetor = (Indice *) malloc(total * sizeof(Indice));

    if (*vetor == NULL) {
        fprintf(stderr, "Erro: falha ao alocar memoria.\n");
        fclose(arq);
        return -1;
    }

    /* segunda passagem */

    rewind(arq);

```

```

fgets(buf, sizeof(buf), arq); /* pula cabecalho */

int i = 0;

int numLinha = 2; /* linha 1 = cabecalho */

while (fgets(buf, sizeof(buf), arq) != NULL && i < total) {

    if (buf[0] == '\n' || buf[0] == '\r') {

        numLinha++;

        continue;

    }

    /* avanca ate a coluna 2 (IBGE7): pula col 0 e col 1 */

    char *tok = strtok(buf, ";"); /* col 0: ConcatUF+Mun */

    tok = strtok(NULL, ";");      /* col 1: IBGE6          */

    tok = strtok(NULL, ";");      /* col 2: IBGE7          */

    if (tok != NULL) {

        (*vetor)[i].ibge = atoi(tok);

        (*vetor)[i].linha = numLinha;

        i++;

    }

    numLinha++;

}

fclose(arq);

printf("Arquivo lido com sucesso.\n");

return i;
}

/* buscaSequencial */

int buscaSequencial(Indice *vetor, int n, int ibge) {

    for (int i = 0; i < n; i++) {

        if (vetor[i].ibge == ibge)

```

```

        return i;

    }

    return -1;
}

/* buscaBinaria */

int buscaBinaria(Indice *vetor, int n, int ibge) {

    int esq = 0;

    int dir = n - 1;

    while (esq <= dir) {

        int meio = esq + (dir - esq) / 2; /* evita overflow */

        if (vetor[meio].ibge == ibge)

            return meio;

        else if (vetor[meio].ibge < ibge)

            esq = meio + 1;

        else

            dir = meio - 1;

    }

    return -1;
}

/* lerLinha */

/* Abre o arquivo, avanca ate numLinha e parseia os campos. */

/* Colunas esperadas: 0=concat 1=ibge6 2=ibge7 3=uf 4=mun 5=reg */

/* 6=pop 7=porte 8=capital */

int lerLinha(const char *nomeArq, int numLinha, Municipio *m) {

    FILE *arq = fopen(nomeArq, "r");

    if (arq == NULL) {

        fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", nomeArq);

        return -1;

    }

```

```
char buf[1024];

for (int i = 1; i < numLinha; i++) {

    if (fgets(buf, sizeof(buf), arq) == NULL) {

        fclose(arq);

        return -1;

    }

}

if (fgets(buf, sizeof(buf), arq) == NULL) {

    fclose(arq);

    return -1;

}

fclose(arq);

buf[strcspn(buf, "\r\n")] = '\0';

char *tok;

tok = strtok(buf, ";"); /* col 0: concat    -- ignora */
tok = strtok(NULL, ";"); /* col 1: ibge6    -- ignora */
tok = strtok(NULL, ";"); /* col 2: ibge7    */
if (tok) m->ibge = atoi(tok);

tok = strtok(NULL, ";"); /* col 3: UF        */
if (tok) strncpy(m->uf, tok, TAM_CAMPO - 1);

tok = strtok(NULL, ";"); /* col 4: Municipio */
if (tok) strncpy(m->municipio, tok, TAM_CAMPO - 1);

tok = strtok(NULL, ";"); /* col 5: Regiao    */
if (tok) strncpy(m->regiao, tok, TAM_CAMPO - 1);

tok = strtok(NULL, ";"); /* col 6: Populacao */
if (tok) m->populacao = atol(tok);
```

```

    tok = strtok(NULL, ";"); /* col 7: Porte */

    if (tok) strncpy(m->porte, tok, TAM_CAMPO - 1);

    return 0;
}

/* exibirMunicipio */

void exibirMunicipio(const Municipio *m) {

    printf(" Municipio : %s\n", m->municipio);

    printf(" UF       : %s\n", m->uf);

    printf(" Regiao    : %s\n", m->regiao);

    printf(" Populacao  : %ld habitantes (Censo 2010)\n", m->populacao);

    printf(" Porte     : %s\n", m->porte);

}

/* medirTempo */

double medirTempo(clock_t inicio, clock_t fim) {

    return (double)(fim - inicio) / CLOCKS_PER_SEC;

}

/* consultarMunicipio */

void consultarMunicipio(Indice *vetor, int n) {

    int ibge;

    printf("Digite o codigo IBGE do municipio (7 digitos): ");

    scanf("%d", &ibge);

    Municipio m;

    clock_t t_inicio, t_fim;

```



```

/* Busca Sequencial */

t_inicio = clock();

int pos_seq = buscaSequencial(vetor, n, ibge);

t_fim = clock();

double tempo_seq = medirTempo(t_inicio, t_fim);


/* Busca Binaria */

t_inicio = clock();

int pos_bin = buscaBinaria(vetor, n, ibge);

t_fim = clock();

double tempo_bin = medirTempo(t_inicio, t_fim);


if (pos_seq == -1) {

    printf("Município com código IBGE %d não encontrado.\n", ibge);

} else {

    if (lerLinha(NOME_ARQUIVO, vetor[pos_seq].linha, &m) == 0) {

        printf("\n--- Dados do Município ---\n");

        exibirMunicípio(&m);

    }

}


printf("\n--- Tempo de execução ---\n");

printf(" Busca Sequencial : %.9f segundos\n", tempo_seq);

printf(" Busca Binaria      : %.9f segundos\n", tempo_bin);


if (pos_seq != pos_bin)

    fprintf(stderr, "Atenção: resultados divergentes entre as buscas.\n");

}


/* liberarMemoria */

void liberarMemoria(Indice **vetor) {

    if (*vetor != NULL) {

```

```
    free(*vetor);

    *vetor = NULL;

}

}
```

## Explicação resumida da solução:

O programa resolve o problema em três camadas separadas de responsabilidade. A primeira é o índice em memória: ao invés de carregar todos os dados de cada município, o programa lê o arquivo CSV duas vezes durante o carregamento. Na primeira passagem apenas conta quantas linhas existem para saber exatamente quanto alocar com malloc. Na segunda passagem preenche o vetor de structs Índice, guardando para cada município apenas o código IBGE7 e o número da linha onde aquele registro está no arquivo. Isso mantém o consumo de memória mínimo e o vetor enxuto, facilitando a busca.

A segunda camada é a busca em si. Como o arquivo já vem ordenado por código IBGE, o vetor carregado também está ordenado, o que permite aplicar busca binária diretamente sem nenhuma ordenação adicional. A busca sequencial percorre o vetor do início ao fim comparando cada código até encontrar ou esgotar os elementos. A busca binária usa dois ponteiros, esquerdo e direito, calculando o meio com a fórmula  $\text{esq} + (\text{dir} - \text{esq}) / 2$  para evitar overflow, e a cada iteração descarta metade dos elementos restantes. Ambas são cronometradas com clock() da biblioteca time.h, e os tempos são exibidos com nove casas decimais para capturar a diferença entre os dois algoritmos.

A terceira camada é a exibição sob demanda. Quando uma busca encontra o município, o programa usa o número de linha guardado no índice para reabrir o arquivo e avançar diretamente até aquela linha, lendo então os campos completos como município, UF, região, população e porte para uma struct Municipio temporária. Essa struct existe apenas durante a exibição e nunca fica alocada no heap, enquanto o vetor de índices é o único dado dinâmico mantido vivo entre consultas. Ao encerrar pela opção 3, free libera esse vetor e o ponteiro é anulado imediatamente, garantindo que nenhum lixo fique na memória.

## TESTES REALIZADOS:

### teste1:

```
rodrigodutra@MacBook-Air-de-Rodrigo learning-journal % cd
"/Users/rodrigodutra/Desktop/learning-journal/EDA1_resolucao_de_exercicios/trabalho4-EDA
```

```
1/" && gcc-15 main.c -o main && ./main
```

```
Algoritmos de Busca - Censo IBGE 2010
```

1. Carregar dados
2. Consultar municipio
3. Sair

```
Opcao: 1
```

```
Arquivo lido com sucesso.
```

```
Dados carregados: 5570 municipios.
```

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 2

Digite o codigo IBGE do municipio: 123

Municipio com codigo IBGE 123 nao encontrado.

--- Tempo de execucao ---

Busca Sequencial : 0.000156000 segundos

Busca Binaria : 0.000001000 segundos

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 2

Digite o codigo IBGE do municipio: 5300108

Municipio com codigo IBGE 5300108 nao encontrado.

--- Tempo de execucao ---

Busca Sequencial : 0.000060000 segundos

Busca Binaria : 0.000002000 segundos

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 2

Digite o codigo IBGE do municipio: 210140

Municipio com codigo IBGE 210140 nao encontrado.

--- Tempo de execucao ---

Busca Sequencial : 0.000060000 segundos

Busca Binaria : 0.000005000 segundos

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados
2. Consultar municipio

3. Sair

Opcao: 2

Digite o codigo IBGE do municipio: 2101400

Municipio com codigo IBGE 2101400 nao encontrado.

--- Tempo de execucao ---

Busca Sequencial : 0.000060000 segundos

Busca Binaria : 0.000002000 segundos

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados

2. Consultar municipio

3. Sair

Opcao: 2

Digite o codigo IBGE do municipio: 83528

Municipio com codigo IBGE 83528 nao encontrado.

--- Tempo de execucao ---

Busca Sequencial : 0.000063000 segundos

Busca Binaria : 0.000002000 segundos

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados

2. Consultar municipio

3. Sair

Opcao: 1

Arquivo lido com sucesso.

Dados carregados: 5570 municipios.

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados

2. Consultar municipio

3. Sair

Opcao: 2

Digite o codigo IBGE do municipio: 3106200

Municipio com codigo IBGE 3106200 nao encontrado.

--- Tempo de execucao ---

Busca Sequencial : 0.000033000 segundos

Busca Binaria : 0.000003000 segundos

Algoritmos de Busca - Censo IBGE 2010

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: ^C

```
rodrigodutra@MacBook-Air-de-Rodrigo trabalho4-EDA1 % cd
"/Users/rodrigodutra/Desktop/learning-journal/EDA1_resolucao_de_exercicios/trabalho4-EDA1/
" && gcc-15 main.c -o main && ./main
```

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 1

Erro: nao foi possivel abrir 'municipios.csv'.

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 2

Carregue os dados primeiro (opcao 1).

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 2

Carregue os dados primeiro (opcao 1).

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 1

Erro: nao foi possivel abrir 'municipios.csv'.

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 3

Memoria liberada. Encerrando.

## teste2:

```
rodrigodutra@MacBook-Air-de-Rodrigo trabalho4-EDA1 % cd  
"/Users/rodrigodutra/Desktop/learning-journal/EDA1_resolucao_de_exercicios/trabalho4-EDA1/  
" && gcc-15 main.c -o main && ./main
```

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 1

Arquivo lido com sucesso.

Dados carregados: 5570 municipios.

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 2

Digite o codigo IBGE do municipio (7 digitos): 2403251

--- Dados do Municipio ---

Municipio : Parnamirim  
UF : RN  
Regiao : Regiao Nordeste  
Populacao : 202456 habitantes (Censo 2010)  
Porte : Grande

--- Tempo de execucao ---

Busca Sequencial : 0.000019000 segundos  
Busca Binaria : 0.000003000 segundos

===== Algoritmos de Busca - Censo IBGE 2010 =====

1. Carregar dados
2. Consultar municipio
3. Sair

Opcao: 3

Memoria liberada. Encerrando.